Selenium Cheat Sheet

Python | Selenium 4 | browser automation & dynamic scraping

1. INSTALL & FIRST SCRIPT

```
# pip install selenium   (4.6+ auto-manages drivers)
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()    # or Firefox(), Edge()
driver.get("https://example.com")
print(driver.title)
driver.quit()                  # close browser + session
```

2. BROWSER OPTIONS

```
from selenium.webdriver.chrome.options import Options

opts = Options()
opts.add_argument("--headless=new")
opts.add_argument("--window-size=1920,1080")
opts.add_argument("--disable-notifications")
opts.add_argument("--user-agent=MyAgent/1.0")
opts.add_experimental_option("detach", True)
driver = webdriver.Chrome(options=opts)
driver.maximize_window()
```

`detach=True` keeps the window open after the script ends. Use `--headless=new` on servers with no display.

3. NAVIGATION & PAGE INFO

`driver.get(url)`	Open URL (waits for page load)
`back()` `forward()` `refresh()`	History and reload
`driver.current_url`	Current URL
`driver.title`	Page title
`driver.page_source`	Full HTML (feed it to BeautifulSoup)
`driver.close()`	Close current tab
`driver.quit()`	Close browser and end session

4. LOCATORS

Strategy	Example
`By.ID`	`find_element(By.ID, "user")`
`By.NAME`	`(By.NAME, "q")`
`By.CLASS_NAME`	`(By.CLASS_NAME, "btn")` single class only
`By.TAG_NAME`	`(By.TAG_NAME, "h1")`
`By.CSS_SELECTOR`	`(By.CSS_SELECTOR, "div.card > a")`
`By.XPATH`	`(By.XPATH, "//a[@id='x']")`
`By.LINK_TEXT`	`(By.LINK_TEXT, "Sign in")`
`By.PARTIAL_LINK_TEXT`	`(By.PARTIAL_LINK_TEXT, "Sign")`

```
el  = driver.find_element(By.ID, "user")
els = driver.find_elements(By.CSS_SELECTOR, "li")
# find_element  -> NoSuchElementException if missing
# find_elements -> [] if nothing matches
```

5. XPATH VS CSS PATTERNS

Goal	XPath	CSS
By id	`//*[@id='a']`	`#a`
By class	`//*[contains(@class,'b')]`	`.b`
Attribute	`//input[@type='text']`	`input[type=text]`
Starts with	`//*[starts-with(@id,'x')]`	`[id^=x]`
Text equals	`//button[text()='Go']`	-
Text contains	`//a[contains(text(),'Go')]`	-
Child	`//div/a`	`div > a`
Descendant	`//div//a`	`div a`
nth item	`(//li)[2]`	`li:nth-child(2)`
Parent	`//span/..`	not possible

Prefer CSS selectors or stable attributes. Avoid long absolute XPaths like `/html/body/div[2]/div[1]` - they break easily.

6. INTERACTING WITH ELEMENTS

`el.click()`	Click
`el.send_keys("text")`	Type into input
`el.clear()`	Clear an input
`el.submit()`	Submit a form
`el.text`	Visible text
`el.get_attribute("href")`	Attribute / property value
`el.is_displayed()`	Visible?
`el.is_enabled()`	Enabled?
`el.is_selected()`	Checked / selected?
`el.value_of_css_property("color")`	Computed CSS value
`el.screenshot("e.png")`	Screenshot of one element
`el.find_element(...)`	Search inside this element

7. KEYBOARD KEYS

```
from selenium.webdriver.common.keys import Keys

box.send_keys("selenium", Keys.ENTER)
box.send_keys(Keys.CONTROL, "a")     # select all
box.send_keys(Keys.TAB)
```

Common keys: ENTER TAB ESCAPE BACKSPACE DELETE SPACE ARROW_DOWN PAGE_DOWN HOME END CONTROL SHIFT ALT F5

8. WAITS (MOST IMPORTANT!)

```
# Implicit: global, applies to every find_element
driver.implicitly_wait(5)

# Explicit: wait for one condition (recommended)
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import (
    expected_conditions as EC)

wait = WebDriverWait(driver, 10)
btn = wait.until(
    EC.element_to_be_clickable((By.ID, "go"))
)
btn.click()
```

EC condition	Waits until
`presence_of_element_located`	Element exists in DOM
`visibility_of_element_located`	In DOM and visible
`element_to_be_clickable`	Visible and enabled
`invisibility_of_element_located`	Gone or hidden

EC condition	Waits until
`presence_of_all_elements_located`	At least one match exists
`text_to_be_present_in_element`	Text appears in element
`title_contains` / `url_contains`	Title / URL has text
`alert_is_present`	An alert is showing
`frame_to_be_available_and_switch_to_it`	Enters the iframe
`number_of_windows_to_be(n)`	n tabs/windows open

Locator arguments are a **tuple**: `EC.presence_of_element_located((By.ID, "x"))` - note the double parentheses. Don't mix implicit and explicit waits.

9. DROPDOWNS (SELECT)

```
from selenium.webdriver.support.ui import Select

sel = Select(driver.find_element(By.ID, "country"))
sel.select_by_visible_text("Egypt")
sel.select_by_value("eg")
sel.select_by_index(2)
print(sel.first_selected_option.text)
print([o.text for o in sel.options])
```

10. MOUSE & ACTIONCHAINS

```
from selenium.webdriver import ActionChains

ac = ActionChains(driver)
ac.move_to_element(menu).perform()     # hover
ac.double_click(el).perform()
ac.context_click(el).perform()         # right-click
ac.drag_and_drop(src, dst).perform()
ac.click_and_hold(el)
ac.move_by_offset(50, 0).release().perform()
ac.send_keys_to_element(el, "hi").perform()
```

11. ALERTS, IFRAMES, TABS

```
# Alerts
alert = driver.switch_to.alert
print(alert.text)
alert.accept()            # OK   (dismiss() = Cancel)
alert.send_keys("text")   # for prompt boxes

# iFrames
driver.switch_to.frame("name_or_id")
driver.switch_to.frame(frame_element)
driver.switch_to.default_content()   # back to main

# Tabs / windows
main = driver.current_window_handle
for h in driver.window_handles:
    driver.switch_to.window(h)
driver.switch_to.new_window("tab")
```

12. JAVASCRIPT, SCREENSHOTS, COOKIES

```
js = "window.scrollTo(0, document.body.scrollHeight)"
driver.execute_script(js)               # scroll bottom
driver.execute_script(
    "arguments[0].scrollIntoView()", el)
driver.execute_script("arguments[0].click()", el)

driver.save_screenshot("page.png")
driver.get_cookies()
driver.add_cookie({"name": "k", "value": "v"})
driver.delete_all_cookies()
```

13. INFINITE SCROLL PATTERN

```
import time
H  = "return document.body.scrollHeight"
GO = "window.scrollTo(0, document.body.scrollHeight)"
last = driver.execute_script(H)
while True:
    driver.execute_script(GO)
    time.sleep(1.5)
    new = driver.execute_script(H)
    if new == last:
        break
    last = new
```

14. COMMON EXCEPTIONS

Exception	Usual cause
`NoSuchElementException`	Wrong locator, or not loaded yet
`TimeoutException`	Wait condition not met in time
`ElementClickInterceptedException`	Popup/overlay covers element
`ElementNotInteractableException`	Element hidden or disabled
`StaleElementReferenceException`	DOM re-rendered; find it again
`NoAlertPresentException`	No alert is open
`WebDriverException`	Driver/browser crashed or mismatch

```
from selenium.common.exceptions import (
    NoSuchElementException, TimeoutException)
```

15. STARTER TEMPLATE

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import (
    expected_conditions as EC)

driver = webdriver.Chrome()
try:
    driver.get("https://quotes.toscrape.com/js/")
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located(
            (By.CLASS_NAME, "quote")))
    for q in driver.find_elements(
            By.CLASS_NAME, "quote"):
        t = q.find_element(By.CLASS_NAME, "text")
        print(t.text)
finally:
    driver.quit()
```

SELENIUM + BEAUTIFULSOUP

```
from bs4 import BeautifulSoup
driver.get(url)
soup = BeautifulSoup(driver.page_source, "lxml")
```

BEST PRACTICES

1	Use explicit waits, not `time.sleep()`
2	Always call `driver.quit()` in a `finally` block
3	Prefer CSS / id / data-* attributes over long XPaths
4	Re-find elements after the page re-renders (stale refs)
5	If the data comes from a JSON API (DevTools > Network), call it directly with `requests` - faster than a browser
6	Respect robots.txt and the site's terms; rate-limit requests